

Del dataset al primer pronóstico

Recopilación de lo aprendido, resultados reales del ejercicio y próximos conceptos para entrenar y comprobar modelos de máximos diarios del SIN.

Estado actual

Sesiones 1 y 2 terminadas. Sesión 3: regresión lineal entrenada y evaluada. Próximo tema propuesto: añadir una variable predictora (máximo de hace siete días).

Objeto del primer ejercicio: pronosticar un único valor: el máximo de demanda eléctrica del día siguiente, expresado en MW. No pronosticamos las 24 horas para luego calcular su máximo.

Materiales de referencia: notebook docente «clase4p1.ipynb», CSV complete-dataset.csv y artículo de descripción del dataset de Morales-Mareco y colaboradores (2025). Los resultados numéricos de este documento proceden de las salidas de Colab compartidas durante el estudio.

Guía de repaso • 22 de septiembre de 2026

Uso: apuntes didácticos; no constituye por sí solo un resultado validado para la tesis.

Ruta de lectura

Bloque	Qué debería poder explicar
1. Objetivo	Qué se predice, cuándo se predice y qué no entrega el modelo.
2. Datos	Qué significa una fila, cuál es la escala temporal y qué está pendiente.
3. Ejemplos X/y	Qué representa shift(1), por qué aparece NaN y qué es un lag.
4. Separación temporal	Por qué usamos entrenamiento y prueba sin barajar fechas.
5. Naive y errores	Cómo hacer una predicción y calcular error, MAE y nMAE.
6. Regresión lineal	Qué aprende fit(), qué produce predict() y qué son a y b.
7. Resultados	Cómo leer los cuatro MAE sin sacar conclusiones indebidas.
8. Próximas sesiones	Variables adicionales, validación, picos y control de fugas.

La película completa

- 1 Plantear el problema: pronosticar el máximo de mañana.
- 2 Preparar datos: de una demanda por hora a un máximo por día.
- 3 Construir ejemplos: entradas conocidas X y objetivo futuro y.
- 4 Reservar fechas posteriores para comprobar pronósticos.
- 5 Probar una referencia simple (naive).
- 6 Entrenar una regresión lineal en el pasado.
- 7 Predecir y comparar ambos sobre las mismas fechas.
- 8 Interpretar, documentar límites y decidir siguiente experimento.

Regla para estudiar

Entender **qué información está disponible en el instante del pronóstico** es tan importante como entender el algoritmo. Entrenar y evaluar son operaciones distintas.

1. ¿Qué queremos pronosticar?

Supongamos que termina el lunes y queremos conocer el máximo del martes. Podemos hacerlo sin predecir todas las horas de ese martes: el modelo produce directamente una sola cantidad (MW).

Enfoque	Salida del modelo	Qué permite saber
Horario	Una predicción para cada hora.	Perfil horario; después podemos tomar el máximo estimado.
Directo diario (nuestro ejercicio)	Un único máximo estimado para mañana.	Magnitud máxima esperada, pero no la hora a la que sucederá.

Definir el instante de emisión

Para este primer problema suponemos que pronosticamos **al finalizar el día actual**, cuando ya conocemos su máximo observado. Si quisiéramos emitir el pronóstico antes de que termine el día, MAX_AYER podría no estar disponible tal como lo definimos. Debemos precisar siempre cuándo emitimos el pronóstico.

Ejemplo

Al finalizar el 1 de enero conocemos su máximo: 1.251,0 MW. Queremos estimar el máximo del 2 de enero, cuyo valor observado en el dataset es 1.440,6 MW.

Vocabulario indispensable

- **Serie temporal:** observaciones ordenadas por fecha y hora; el orden importa.
- **Horizonte de pronóstico:** cuánto tiempo hacia delante queremos predecir. Aquí, un día.
- **Variable objetivo (target):** máximo diario que queremos pronosticar.
- **Variable predictora (feature):** información suministrada al modelo antes del día objetivo.
- **Ejemplo o muestra:** una fila que empareja entradas conocidas con el resultado real correspondiente.

2. Datos y preprocesamiento realizado

Archivo de partida

Elemento	Valor observado
Archivo	complete-dataset.csv
Tamaño original	131.481 filas y 9 columnas
Campos principales	DATETIME y SIN (demanda en MW)
Otros campos	Meteorología y variable de feriado (ISHOLIDAY)
Zona horaria	Fechas originales con offset histórico -03:00 / -04:00

Creamos DATETIME.UTC con `pd.to_datetime(..., utc=True)` y DATETIME_LOCAL mediante `tz_convert("America/Asuncion")`. Son dos formas de representar el mismo instante. La fecha local es la que corresponde usar para agrupar observaciones por día paraguay.

Limpieza ejecutada

```
df["DATETIME.UTC"] = pd.to_datetime(df["DATETIME"], utc=True)
df["DATETIME_LOCAL"] = (
    df["DATETIME.UTC"].dt.tz_convert("America/Asuncion")
)
df = df.drop_duplicates().sort_values("DATETIME.UTC")
df = df.reset_index(drop=True)
```

Se identificaron **15 filas completamente duplicadas** y se eliminaron. Quedaron **131.466 filas**. También se detectaron **15 saltos de tres horas en UTC**; cada salto implica dos posiciones horarias no presentes en una cuadrícula UTC uniforme.

Lo que afirma el artículo y lo que no queda cerrado

El artículo de origen declara que el CSV procesado incluye huecos intencionales asociados a cambios estacionales de horario en Paraguay (p. 3). En un caso observado, la secuencia salta de 08/03/2008 22:00 -03:00 (01:00 UTC) a 09/03/2008 00:00 -04:00 (04:00 UTC). Falta comprobar en el archivo fuente el mecanismo exacto que produjo cada ausencia; la descripción general del artículo no reconstruye esos pares ni documenta expresamente los 15 duplicados.

Decisión metodológica pendiente

No imputamos automáticamente los dos registros ausentes de cada salto: un valor inventado puede ocultar el verdadero máximo diario. La cobertura de los días afectados debe revisarse antes de dar por válidos los resultados de tesis.

3. De horas a máximos diarios

Al agrupar las observaciones por FECHA local, cada fila pasa a representar un día. El máximo calculado es el **máximo observado en las horas disponibles**, no necesariamente un máximo validado si hay registros ausentes.

```
df["FECHA"] = df["DATETIME_LOCAL"].dt.date
df_diario = (
    df.groupby("FECHA")
      .agg(SIN_max=("SIN", "max"),
          N_REGISTROS=("SIN", "count"))
      .reset_index()
)
```

FECHA	SIN_max (MW)	N_REGISTROS
2008-01-01	1.251,0	24
2008-01-02	1.440,6	24
2008-01-03	1.458,0	24

Resultado obtenido: 5.479 fechas diarias. N_REGISTROS cuenta valores no nulos de SIN por día; conservarlo nos ayuda a revisar cobertura, pero 24 registros por sí solos no demuestran que todas las horas esperadas sean correctas ni que el máximo sea verdadero.

Ahora construimos el lag de un día

```
df_diario = df_diario.sort_values("FECHA").reset_index(drop=True)
df_diario["MAX_AYER"] = df_diario["SIN_max"].shift(1)
```

shift(1) desplaza la columna una fila: en la fila de hoy aparece el máximo del registro diario anterior.

No verifica las fechas. Por eso comprobamos que cada pareja corresponde a días consecutivos:

```
df_diario["FECHA"] = pd.to_datetime(df_diario["FECHA"])
es_consecutivo = (
    df_diario["FECHA"].diff() == pd.Timedelta(days=1)
)
df_diario.loc[~es_consecutivo, "MAX_AYER"] = float("nan")
```

¿Qué hicimos con el primer NaN?

El primer día no tiene MAX_AYER: no disponemos del máximo de 31/12/2007. No inventamos un valor ni lo sustituimos por cero; excluimos ese par del conjunto de ejemplos. **df_diario conserva la fila;** una nueva tabla datos_modelo contiene solo pares válidos.

```

datos_modelo = df_diario.dropna(
    subset=["MAX_AYER", "SIN_max"]
)
X = datos_modelo[["MAX_AYER"]] # entradas: DataFrame
y = datos_modelo["SIN_max"]   # objetivo: Series

```

Resultado: 5.478 pares utilizables en este ejercicio. Nota: esto elimina el NaN de un lag no disponible; no resuelve los huecos horarios de origen.

4. ¿Cómo dividimos los ejemplos?

Los ejemplos se mantienen ordenados: primero el período antiguo para entrenamiento; después el período más reciente para prueba. No barajamos fechas porque queremos simular que el futuro aún no ocurrió.

```

datos_modelo = datos_modelo.sort_values("FECHA")
n_train = int(len(datos_modelo) * 0.8)
train = datos_modelo.iloc[:n_train]
test = datos_modelo.iloc[n_train:]

```

Conjunto	Ejemplos	Uso
Entrenamiento	4.382	Aprender parámetros a partir del pasado.
Prueba	1.096	Comprobar predicciones sobre fechas posteriores.

El conjunto de prueba comienza el **01/01/2020** y abarca el período final del dataset. La primera entrada de prueba usa el máximo real del día anterior; esto es válido si emitimos pronósticos diarios de un paso y el valor de ayer ya está registrado.

Tres verbos que no deben confundirse

Operación	Pregunta	Ejemplo
Entrenar	¿Qué aprende?	<code>fit(X_train, y_train)</code>
Predecir	¿Qué estima?	<code>predict(X_test)</code>
Evaluar	¿Cuánto se equivoca?	<code>MAE(y_test, predicción)</code>

Por qué separamos datos

Un modelo puede ajustarse a su pasado sin funcionar igual de bien con datos posteriores. La prueba mide ese comportamiento fuera del entrenamiento. Más adelante necesitaremos también **validación** para elegir variables e hiperparámetros sin utilizar el test para tomar todas las decisiones.

5. Primer modelo de referencia: naive

Antes de usar un algoritmo que aprende, probamos una regla fija: **el máximo de mañana será igual al máximo de hoy**. Eso es un pronóstico naive; no tiene etapa de ajuste.

Pronóstico naive: $\hat{y}(t) = y(t - 1) = \text{MAX_AYER}$

```
pred_naive = test["MAX_AYER"]
resultados_naive = test[["FECHA", "SIN_max"]].copy()
resultados_naive["PREDICCION"] = pred_naive
```

Fecha	Real (MW)	Naive (MW)
2020-01-01	2.492,0	3.010,0
2020-01-02	2.762,0	2.492,0
2020-01-03	2.749,0	2.762,0

Error de predicción: fijamos una convención de signos

Error = real – pronosticado = $y - \hat{y}$

Si error > 0, **subestimamos**; si error < 0, **sobreestimamos**. El 01/01/2020: $2.492 - 3.010 = -518$ MW; el naive sobreestimó por 518 MW.

El **error absoluto** es $|\text{error}|$: solo indica la magnitud, independientemente del signo. Así impedimos que errores de +100 MW y -100 MW se cancelen al promediar.

MAE: promedio de los errores absolutos

MAE = $(1 / n) \times \sum |y(i) - \hat{y}(i)|$

Sobre **1.096 días de prueba**, obtuvimos MAE naive = **188,60 MW**. No indica que el error sea 188,60 MW cada día ni distingue dirección del error.

6. Escala del error y primeras métricas

El máximo diario promedio durante la prueba fue **2.662,36 MW**. Para expresar el MAE en relación con esa escala, calculamos:

$$\text{nMAE} = \text{MAE} / \text{promedio del máximo real} \times 100$$

Resultado: $188,60 / 2.662,36 \times 100 = 7,08 \%$ (redondeado). Este nMAE usa un denominador único, calculado sobre la prueba; **no es MAPE**, que promedia errores porcentuales calculados día por día.

```
resultados_naive["ERROR"] = (  
    resultados_naive["SIN_max"] - resultados_naive["PREDICCION"]  
)  
resultados_naive["ERROR_ABSOLUTO"] = (  
    resultados_naive["ERROR"].abs()  
)  
mae_naive = resultados_naive["ERROR_ABSOLUTO"].mean()  
promedio_test = test["SIN_max"].mean()  
nmae = 100 * mae_naive / promedio_test
```

Tres preguntas distintas

Métrica o columna	Responde
ERROR (con signo)	¿El modelo subestimó o sobreestimó ese día?
Error absoluto	¿Cuánto se equivocó ese día en MW?
MAE	¿Cuánto se equivoca en promedio absoluto?
nMAE	¿Qué representa el MAE frente al promedio de demanda?

Regla de comparación

Un MAE por sí solo no permite decir si el modelo es suficientemente preciso. Primero debemos establecer una referencia, utilizar el mismo período de prueba y considerar los objetivos operativos del pronóstico.

7. Nuestra primera regresión lineal

La regresión lineal recibe MAX_AYER y aprende una transformación con **dos parámetros**: intercepto a y pendiente b. A diferencia del naive, no fija los parámetros de antemano.

$$\text{Pronóstico lineal: } \hat{y} = a + b \times X$$

```
from sklearn.linear_model import LinearRegression
X_train = train[["MAX_AYER"]]
y_train = train["SIN_max"]
modelo_lineal = LinearRegression()
modelo_lineal.fit(X_train, y_train)
print(modelo_lineal.intercept_, modelo_lineal.coef_[0])
```

Parámetro aprendido	Valor	Interpretación
Intercepto (a)	176,7815	Término constante. X=0 no es un escenario típico de nuestra demanda.
Pendiente (b)	0,9080545	Al aumentar X en 100 MW, la predicción aumenta ~90,81 MW.

$$\text{Ecuación aprendida: } \hat{y} = 176,78 + 0,9081 \times \text{MAX_AYER}$$

¿Qué hace realmente fit()?

LinearRegression ajusta a y b para minimizar la **suma de errores cuadrados** en entrenamiento (mínimos cuadrados ordinarios). Esto penaliza más los errores grandes. No minimiza directamente el MAE de prueba. El algoritmo puede completar el entrenamiento sin garantizar menor MAE posterior.

¿Qué hace predict()?

```
X_test = test[["MAX_AYER"]]
pred_lineal = modelo_lineal.predict(X_test)
```

Aplica la relación aprendida a cada entrada del conjunto de prueba. No vuelve a ajustar a ni b. Para 01/01/2020, la predicción fue aproximadamente **2.910,03 MW**, frente a 3.010 MW del naive y 2.492 MW reales.

8. Evaluación: los cuatro resultados reales

Comparamos sobre las mismas fechas y utilizando la misma métrica: MAE. También medimos el período de entrenamiento para estudiar cuánto se transfiere el desempeño a observaciones posteriores.

Modelo	MAE entrenamiento	MAE prueba
Naive	129,39 MW	188,60 MW
Regresión lineal	129,11 MW	195,55 MW

Lectura responsable

- En **entrenamiento**, la regresión lineal reduce el MAE frente al naive solo **0,28 MW**. Es una mejora muy pequeña para esta configuración.
- En **prueba**, la regresión lineal registra un MAE **6,95 MW mayor** que el naive. Por tanto, la mejora observada al entrenar no se sostuvo en el período posterior.
- Ambos modelos presentan mayor MAE en prueba que en entrenamiento. Esto **no demuestra por sí solo sobreajuste**: también puede deberse a cambios en niveles de demanda, composición de días, condiciones distintas o episodios difíciles.
- Una relación aprendida para minimizar errores cuadrados en entrenamiento no tiene por qué minimizar MAE en prueba.

¿Para qué sirvió medir el MAE de entrenamiento?

Permite responder cuánto error presentan los modelos en el período que vio la regresión. Al contrastarlo con el MAE de prueba entendemos la **generalización**: rendimiento sobre datos no usados para aprender. La referencia naive no se entrena, pero podemos medir su error en ambos períodos para comparar en condiciones equivalentes.

Conclusión limitada pero válida

En este ejercicio didáctico, con una sola variable (MAX_AYER), el naive obtuvo menor MAE en prueba que la regresión lineal. **No** hemos demostrado que la regresión lineal sea siempre inferior ni que alguno pronostique mejor los picos extremos.

9. Qué queda pendiente antes de convertirlo en evidencia de tesis

A. Calidad y definición de los datos

- Revisar y documentar la cobertura de los días afectados por las discontinuidades temporales; conservar la distinción entre máximo observado y máximo plenamente validado.
- Precisar fecha local, instantes UTC y tratamientos de registros repetidos; contrastar con las fuentes originales cuando sea necesario.
- Revisar qué limpiezas e imputaciones ya trae el dataset publicado; evitar presentar valores estimados como observaciones originales.

B. Diseño del experimento

- Fijar el instante de emisión: aquí asumimos pronóstico diario al terminar el día previo.
- Separar entrenamiento, validación y prueba. Seleccionar variables o configuraciones con validación cronológica, no utilizando repetidamente la prueba final.
- Para cada variable, demostrar que estaría disponible en el instante de pronóstico: el tiempo real del día siguiente no puede usarse como entrada salvo que sea un **pronóstico meteorológico disponible** en ese momento.
- Comparar sobre idénticas fechas, idéntica variable objetivo y métricas apropiadas.

C. Particularidades de los picos

Pronosticar el **máximo diario** no equivale a localizar su **hora**. Un modelo que devuelve un único máximo no permite calcular por sí solo métricas de coincidencia de hora pico ni reconstruir el perfil horario.

Para la tesis, además de MAE y RMSE, definiremos formalmente los eventos de alta demanda (máximos, percentiles, ventanas) y estudiaremos frecuencia y magnitud de subestimaciones y sobreestimaciones. Cualquier métrica como Peak Recall requiere especificar qué cuenta como evento y qué cuenta como acierto.

Fuente del dataset

Morales-Mareco, F. y cols. (2025). Electricity demand from national interconnected system of Paraguay and meteorological dataset. Data in Brief, 61, 111782. DOI: 10.1016/j.dib.2025.111782. Cambios horarios: p. 3; proceso de imputación y tratamiento de atípicos: pp. 7-8. El paper describe el procesamiento publicado, pero no documenta con detalle el origen de cada una de nuestras 15 filas duplicadas y 15 discontinuidades UTC.

10. Próximas sesiones: ruta de aprendizaje

Ya completamos una iteración entera: datos → ejemplos → referencia → entrenamiento → predicción → evaluación. No hace falta introducir muchos algoritmos ahora. Lo siguiente es variar **una sola cosa cada vez**.

Próximo bloque	Qué aprenderemos / comprobaremos
Sesión 4: nueva entrada	Añadir MAX_7_DIAS y explicar qué aporta frente a MAX_AYER. Respetar fechas, NaN y disponibilidad.
Sesión 5: más entradas	Incorporar, por separado, calendario y luego clima disponible al pronosticar. Comprender variables predictoras.
Sesión 6: evaluación robusta	Separación train/validación/test y validación temporal; por qué no optimizar usando test final.
Sesión 7: diagnóstico	Gráficos real vs. predicho, errores, estacionalidad, sesgo, cambios de escala, sobreajuste.
Sesión 8: picos	Identificar picos y cuantificar errores condicionados a eventos de alta demanda.
Sesión 9: alternativas	Comparar otros modelos o estrategias especializadas solo después de fijar una evaluación reproducible.

Siguiente cambio propuesto: MAX_7_DIAS

Un lag de 7 días aporta el máximo del mismo día de la semana anterior. Es una hipótesis para capturar regularidad semanal, **no una garantía de mejora**. Su incorporación cambiará qué días tienen todas las entradas disponibles; habrá que recalcular conjuntos comparables y mantener intacta la prueba final si queremos una comparación confirmatoria.

Miniglosario para recordar

Término	Definición breve
Lag	Valor anterior de una serie, desplazado k períodos.
Feature / X	Información conocida que recibe el modelo.
Target / y	Valor real que intentamos predecir.
fit()	Ajusta los parámetros a partir del entrenamiento.
predict()	Calcula predicciones sin volver a entrenar.
MAE	Promedio de los errores absolutos.
Generalización	Desempeño en observaciones no utilizadas para entrenar.

11. Autoevaluación y punto exacto de continuación

Respondé sin mirar el código

- 1. Si pronostico el máximo del martes al finalizar el lunes, ¿qué representa X y qué representa y en el modelo de una variable?
- 2. ¿Por qué el primer día recibió MAX_AYER = NaN? ¿Por qué no lo reemplazamos por cero?
- 3. ¿Por qué separamos el período final como prueba y no mezclamos aleatoriamente las filas?
- 4. ¿Qué aprende la regresión lineal mediante fit()? ¿Qué hace predict()?
- 5. Si el error real - pronosticado es negativo, ¿se subestimó o sobreestimó?
- 6. ¿Por qué el MAE de entrenamiento y el de prueba pueden ser diferentes?
- 7. ¿Un MAE bajo garantiza buen pronóstico de los picos extremos? ¿Por qué no?

Respuestas breves

1) X = máximo conocido del día anterior; y = máximo real del día objetivo. 2) Falta un día anterior en el archivo: no inventamos el dato. 3) Para simular futuro no visto. 4) Aprende intercepto y pendiente; predict() aplica esos parámetros. 5) Sobreestimación. 6) Son fechas distintas y solo train intervino en el ajuste. 7) No: el promedio puede ocultar errores grandes en días críticos.

Dónde continuar en el próximo encuentro

Cuaderno: **03_Primer_Modelo_Maximo_Diario.ipynb**. Ya existen df, df_diario, datos_modelo, train, test, pred_naive, modelo_lineal y pred_lineal. Última comparación: MAE naive = 188,60 MW y MAE lineal = 195,55 MW en prueba. Próximo tema: comprender y construir MAX_7_DIAS, sin añadir otras variables aún.

Recordatorio: guardar el notebook y anotar las transformaciones y los supuestos; en Colab, archivos cargados al entorno de ejecución pueden desaparecer al reiniciar la sesión.